BCA-302: Theory of Computing (Important Q&A)

Weightage: 70 (Block I: 30, Block II: 40) Marks

Q1. What is a Regular Expression?

A regular expression (RE) is a sequence of characters that defines a search pattern. In Automata theory, it is an algebraic way to describe languages that can be accepted by Finite Automata.

Mathematical/Theoretical Expression: R = a*b + c

Q2. Define Kleene Closure.

The Kleene closure (or star operator) of a set V, denoted V*, is the set of all strings over V of any length, including the empty string (epsilon).

Mathematical/Theoretical Expression: V* = V^0 U V^1 U V^2 U ...

Q3. What are Regular Languages?

A regular language is a formal language that can be expressed using a regular expression and accepted by a deterministic or non-deterministic finite automaton.

Mathematical/Theoretical Expression: L(R) = {w | w matches R}

Q4. Define a Finite Automaton (FA).

A finite automaton is a mathematical model of computation consisting of a finite set of states, a set of input symbols, a transition function, a start state, and a set of accept states.

Mathematical/Theoretical Expression: M = (Q, Σ, δ, q0, F)

Q5. Differentiate between DFA and NFA.

In a Deterministic Finite Automaton (DFA), for every state and input symbol, there is exactly one next state. In a Non-Deterministic Finite Automaton (NFA), there can be zero, one, or multiple next states for a given input.

Mathematical/Theoretical Expression: DFA: δ(q, a) = p | NFA: δ(q, a) = {p1, p2...}

Q6. What are Finite Automata with Output?

Standard FA only accept or reject strings. FA with output (like Moore and Mealy machines) produce a specific output sequence for an input sequence.

Mathematical/Theoretical Expression: Mealy: Output depends on state AND input

Q7. Explain the Equivalence of NFA and DFA.

Every language accepted by an NFA can also be accepted by a DFA. The NFA can be converted to an equivalent DFA using the Subset Construction algorithm, though the DFA may have up to 2^N states.

Mathematical/Theoretical Expression: L(NFA) = L(DFA)

Q8. State the Pumping Lemma for Regular Languages.

The Pumping Lemma is a property that all regular languages must satisfy. It is primarily used to prove that a specific language is NOT regular by demonstrating it cannot be 'pumped' without leaving the language.

Mathematical/Theoretical Expression: $w = xyz$, $|xy| \leq p$, $|y| \geq 1$, $xy^iz \in L$

Q9. What are the Closure Properties of Regular Languages?

Regular languages are closed under operations like Union, Intersection, Concatenation, Kleene Star, and Complementation. This means applying these operations to regular languages yields another regular language.

Mathematical/Theoretical Expression: $L1 \cup L2$ is regular if L1, L2 are regular

Q10. Define Context-Free Grammar (CFG).

A CFG is a formal grammar consisting of a set of non-terminal symbols, terminal symbols, production rules, and a start symbol. Every production rule replaces a single non-terminal with a string of terminals and/or non-terminals.

Mathematical/Theoretical Expression: $G = (V, T, P, S)$ | Rule: $A \rightarrow \alpha$

Q11. Explain the Chomsky Hierarchy.

The Chomsky Hierarchy categorizes formal grammars into four levels: Type 0 (Unrestricted / Turing Machine), Type 1 (Context-Sensitive / LBA), Type 2 (Context-Free / PDA), and Type 3 (Regular / FA).

Mathematical/Theoretical Expression: Type 3 ⊂ Type 2 ⊂ Type 1 ⊂ Type 0

Q12. What is a Pushdown Automaton (PDA)?

A PDA is a finite automaton equipped with a stack (LIFO memory). This extra memory allows it to recognize Context-Free Languages, such as matching parentheses, which a standard FA cannot do.

Mathematical/Theoretical Expression: PDA = FA + Stack

Q13. Discuss the Equivalence of CFG and PDA.

For every Context-Free Grammar, there exists a Pushdown Automaton that accepts the same language, and vice versa. They are theoretically equivalent in computational power.

Mathematical/Theoretical Expression: L(CFG) = L(PDA)

Q14. What are Non-Context Free Languages?

These are languages that cannot be generated by a CFG. For example, the language L = {a^n b^n c^n | n ≥ 1} requires tracking three identical counts, which a single stack (PDA) cannot manage.

Mathematical/Theoretical Expression: L = {a^n b^n c^n} is not CFL

Q15. State the Pumping Lemma for Context-Free Languages.

Similar to the regular pumping lemma, this lemma states that any sufficiently long string in a CFL can be divided into five parts (uvwxy) such that 'v' and 'y' can be pumped simultaneously.

Mathematical/Theoretical Expression: w = uvwxy, |vwx| ≤ p, |vy| ≥ 1, uv^iwx^iy ∈ L

Q16. Define a Turing Machine (TM).

A Turing Machine is a mathematical model of computation that defines an abstract machine manipulating symbols on an infinite tape according to a table of rules. It is the theoretical foundation of modern computers.

Mathematical/Theoretical Expression: $M = (Q, \Gamma, b, \Sigma, \delta, q0, F)$

Q17. Explain the Transition Diagram of a Turing Machine.

A transition diagram for a TM is a directed graph where nodes are states. Edges represent transitions and are labeled with (read_symbol, write_symbol, move_direction).

Mathematical/Theoretical Expression: Edge Label: a / b, R (read a, write b, move Right)

Q18. What is the difference between a language 'Accepted' and 'Decided' by a TM?

A TM 'accepts' a language if it halts in an accept state for all strings in the language (Turing-Recognizable). It 'decides' a language if it halts on ALL inputs, accepting strings in the language and rejecting strings not in the language (Turing-Decidable).

Mathematical/Theoretical Expression: Decidable implies halting on all inputs

Q19. How do you construct a Turing Machine?

Constructing a TM involves defining the set of states and formulating a transition function that dictates how the read/write head moves along the infinite tape to process the input logic step-by-step.

Mathematical/Theoretical Expression: $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

Q20. Explain the Halting Problem conceptually.

The Halting Problem asks whether it is possible to create a general algorithm that, given a program and its input, determines if the program will eventually stop or run forever. Alan Turing proved this is undecidable.

Mathematical/Theoretical Expression: No algorithm can solve the Halting Problem for all inputs.